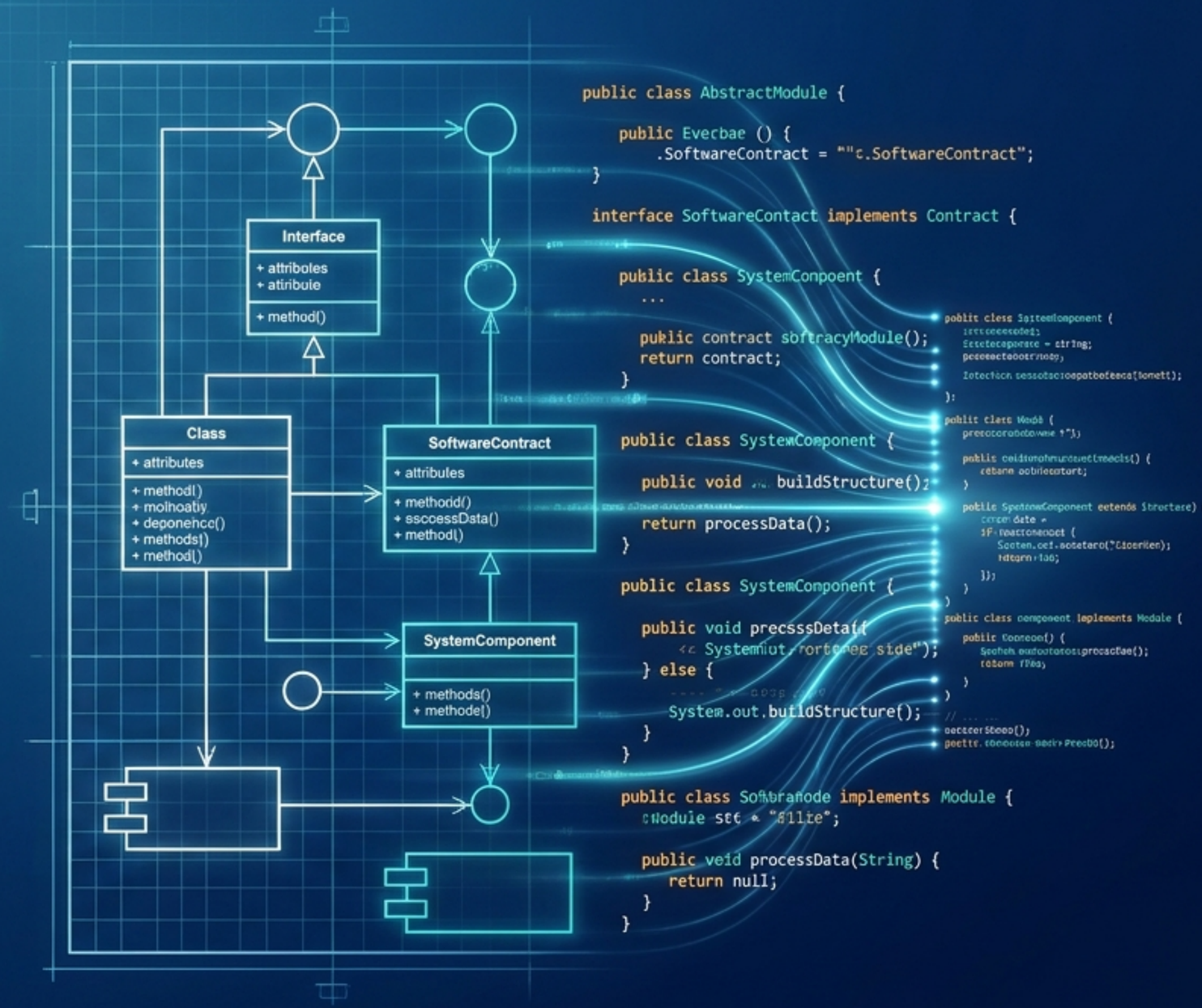
Arquitetura de Software em Java

Modelando abstrações, contratos e sistemas com notação UML

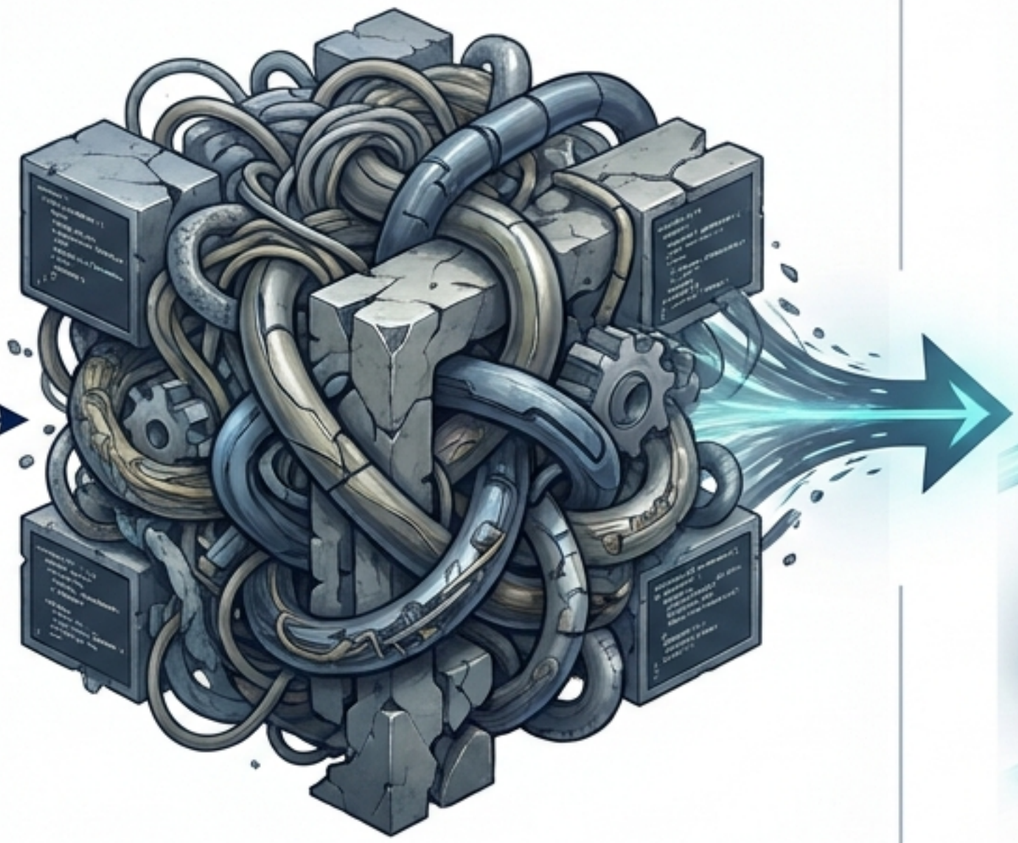


Gemini Notebook

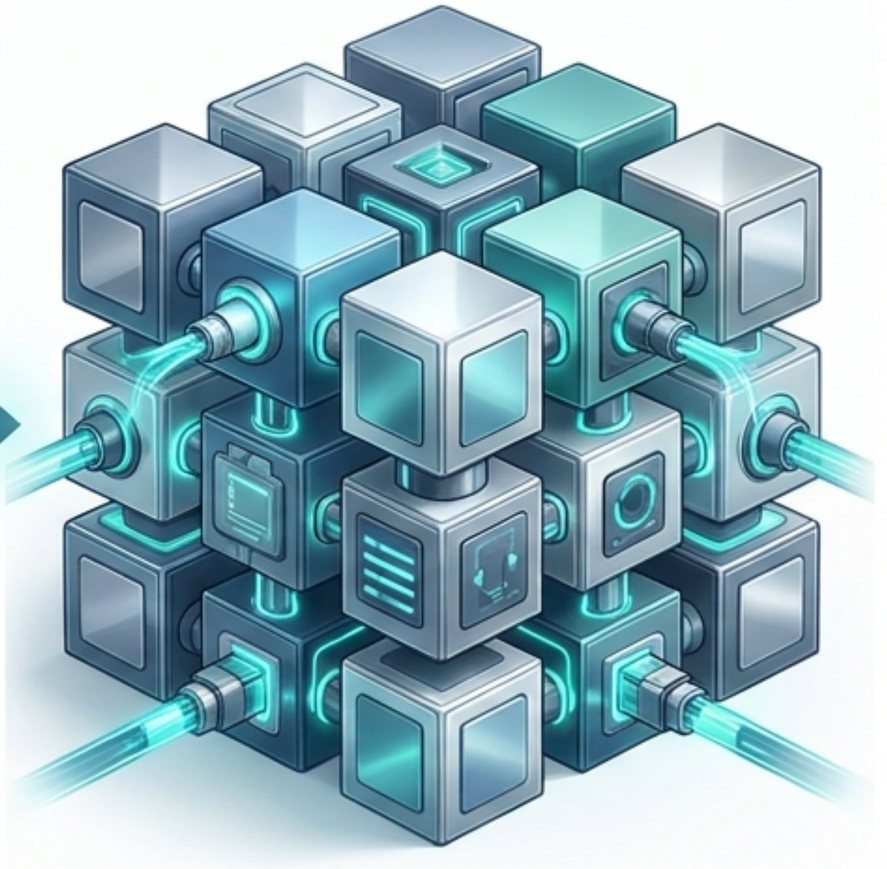
O Desafio do Acoplamento e da Rigidez

Rigidez

Sistemas quebram quando componentes dependem excessivamente de implementações concretas.



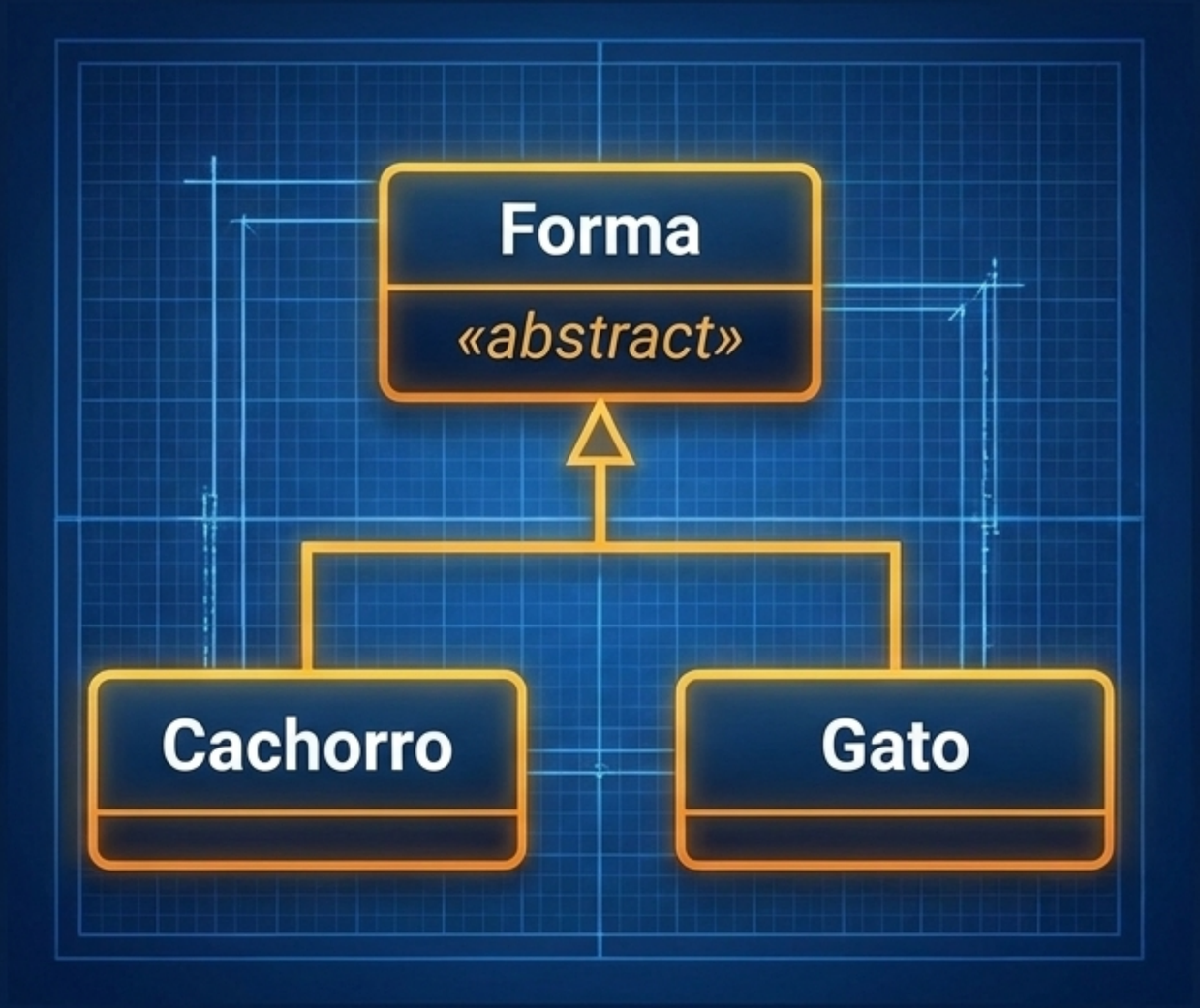
Modularidade



O bom design de software foca no *que* o objeto faz (comportamento), não em *como* ele faz (implementação).

Gemini Notebook

Classes Abstratas (*abstract*): A Relação "É-Um"



Identidade e Estado

Permite definir atributos de estado contínuo (variáveis de instância).

Reuso e Obrigação

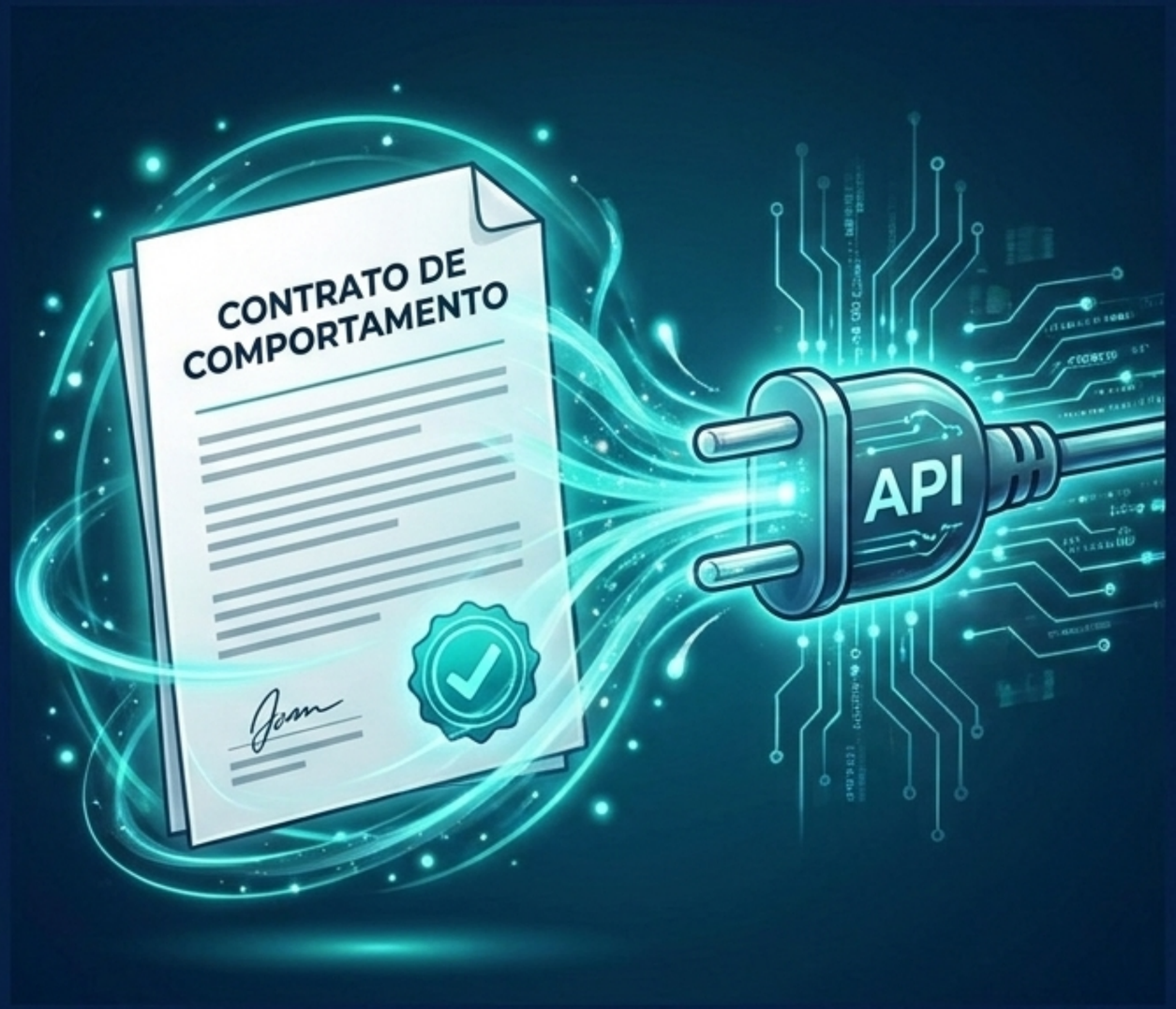
Mistura métodos concretos (reaproveitados) e métodos abstratos (que forçam a implementação nas subclasses).

Regra de Ouro

Não pode ser instanciada diretamente (o uso de 'new' é proibido).

Gemini Notebook

Interfaces (*interface*): A Relação "É-Capaz-De"



Contratos de Comportamento

Define papéis ou capacidades (ex: Atendimento, Autenticavel), sem se importar com a identidade da classe.

Ausência de Estado

Não possui variáveis de instância, apenas constantes (public static final).

Evolução (Java 8+)

Permite métodos 'default' para adicionar novos comportamentos sem quebrar implementações existentes.

Flexibilidade Máxima

Uma classe pode implementar múltiplas interfaces simultaneamente.

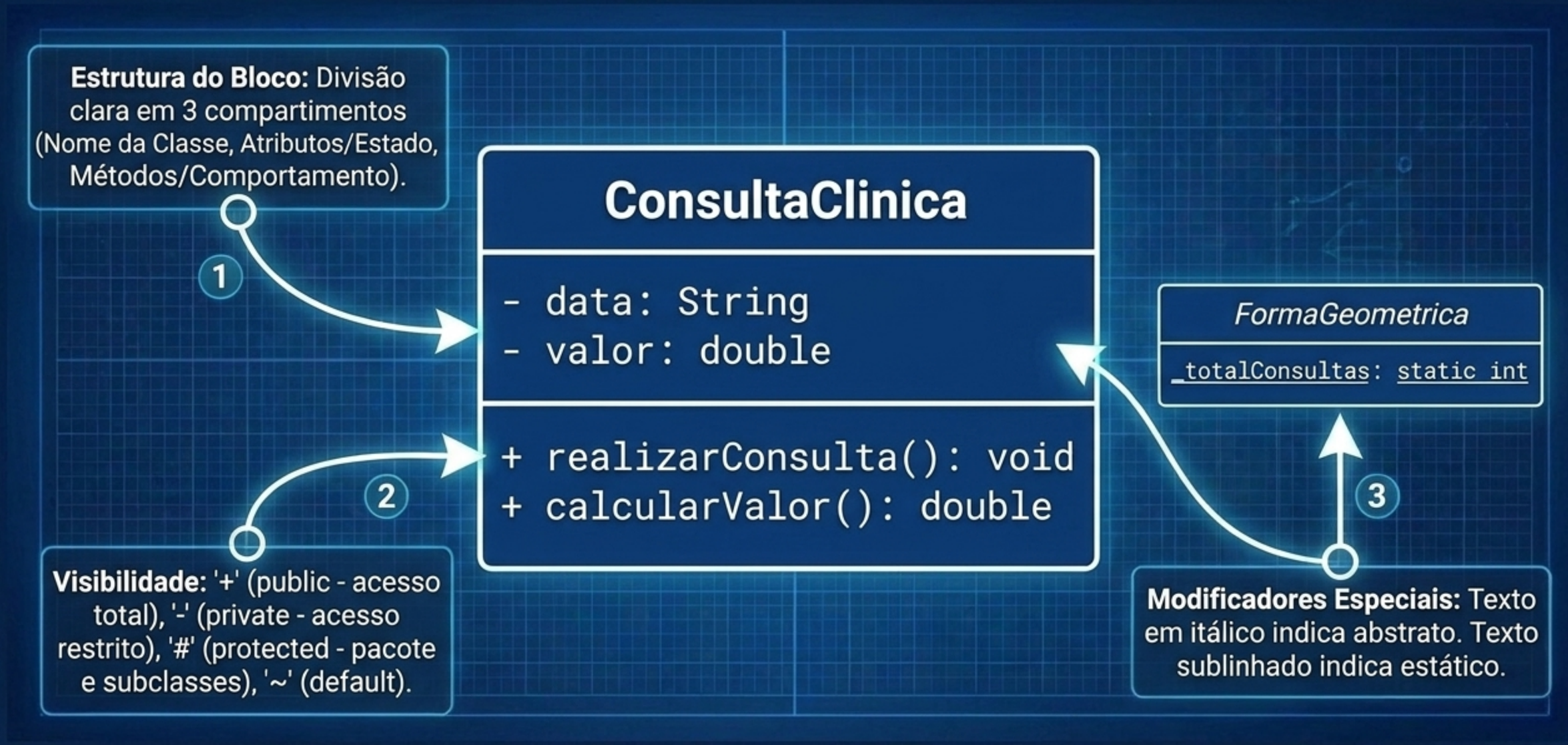
Gemini Notebook

O Quadro Comparativo: Estado vs. Contrato

	Classes Abstratas	Interfaces
Herança	Única (limita a árvore de herança)	Múltipla (flexibilidade total)
Estado	Mantém estado (variáveis de instância)	Sem estado (apenas constantes)
Métodos	Concretos e Abstratos	Abstratos e Default (Java 8+)
Acoplamento	Forte (relação estrutural e de identidade)	Fraco (relação de papel e serviço)

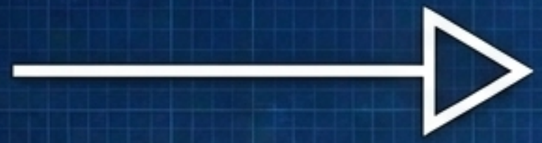
Gemini Notebook

O Vocabulário Visual: Anatomia de uma Classe UML

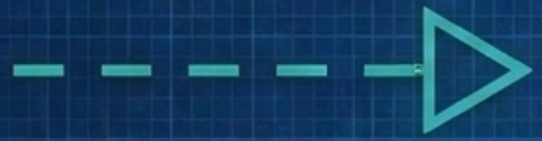


Gemini Notebook

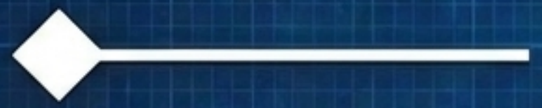
A Gramática UML: Conectando o Sistema



Herança (extends): A relação forte 'É-um'.



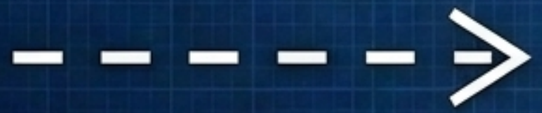
Realização (implements): Adoção de um contrato.



Composição: 'Tem-um' forte. A parte nasce e morre com o todo (ex: Consulta e Prontuário).



Agregação: 'Tem-um' fraco. A parte tem vida independente (ex: Consulta e Tutor).

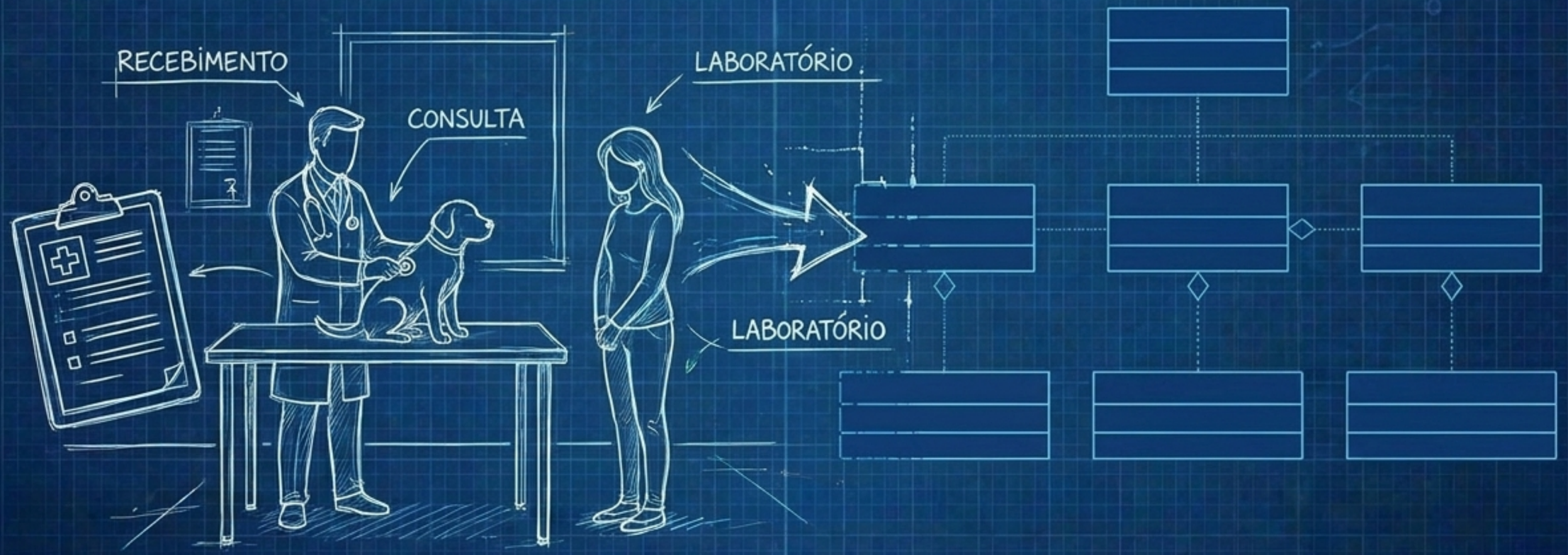


Dependência: 'Usa-um' temporário, geralmente via parâmetro.

Gemini Notebook

Estudo de Caso: O Sistema da Clínica Veterinária

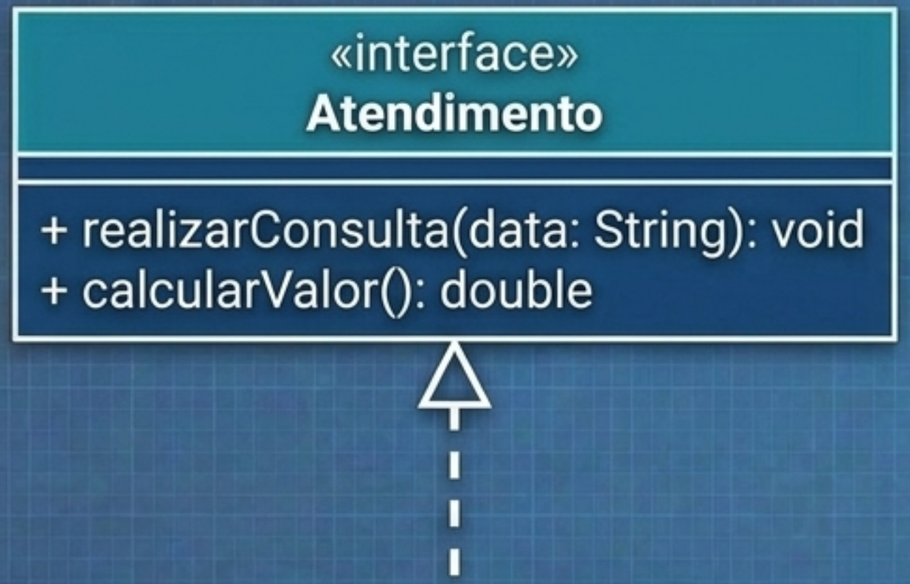
Precisamos modelar um sistema que integre atendimentos financeiros, taxonomias de animais, e registros médicos sem criar um código rígido e inseparável.



Gemini Notebook

Definindo Contratos: A Interface Atendimento

Design



Implementação

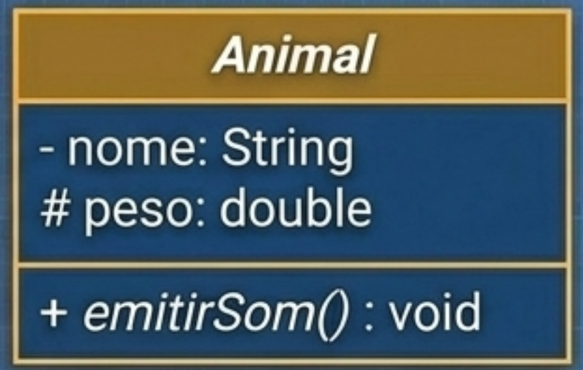
```java
public interface Atendimento {
    void realizarConsulta(String data);
    double calcularValor();
}
```

O contrato garante que qualquer classe de serviço da clínica saberá como calcular valores e registrar consultas.

Gemini Notebook

Modelando a Taxonomia: A Classe Abstrata Animal

Design



Implementação

```java
public abstract class Animal {
    private String nome;
    protected double peso;

    // Construtor e getters/setters omitidos

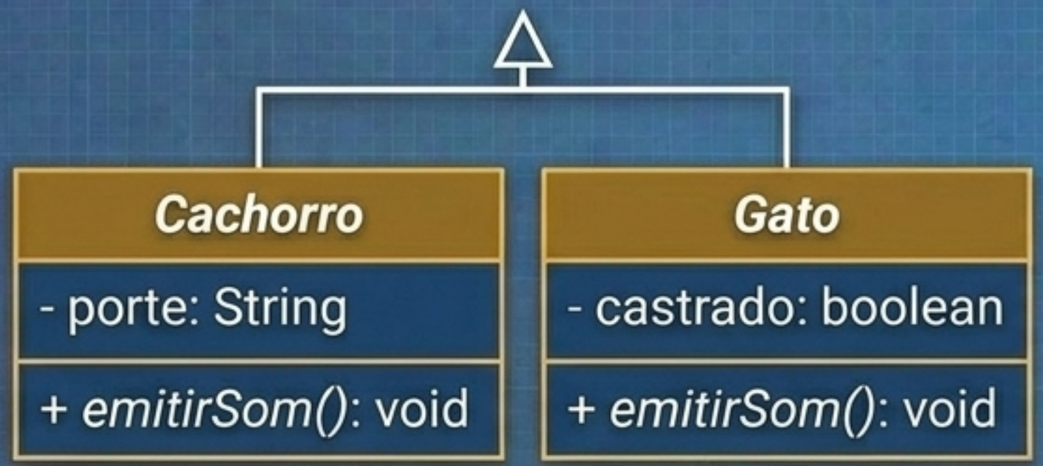
    public abstract void emitirSom();
}
```

Encapsula o estado comum (nome, peso), mas obriga as subclasses a definirem seu próprio comportamento acústico.

Gemini Notebook

Especialização e Implementação Concreta

Design



Implementação

```java
public class Cachorro extends Animal {
    private String porte;

    @Override
    public void emitirSom() {
        System.out.println("Latido");
    }
}
```

As subclasses herdam a estrutura física, mas concretizam a identidade.

Gemini Notebook

Estruturas de Posse: Composição vs. Agregação

Composição



Composição (Morte com o todo): O prontuário não existe no sistema sem uma consulta que o origine.

Agregação



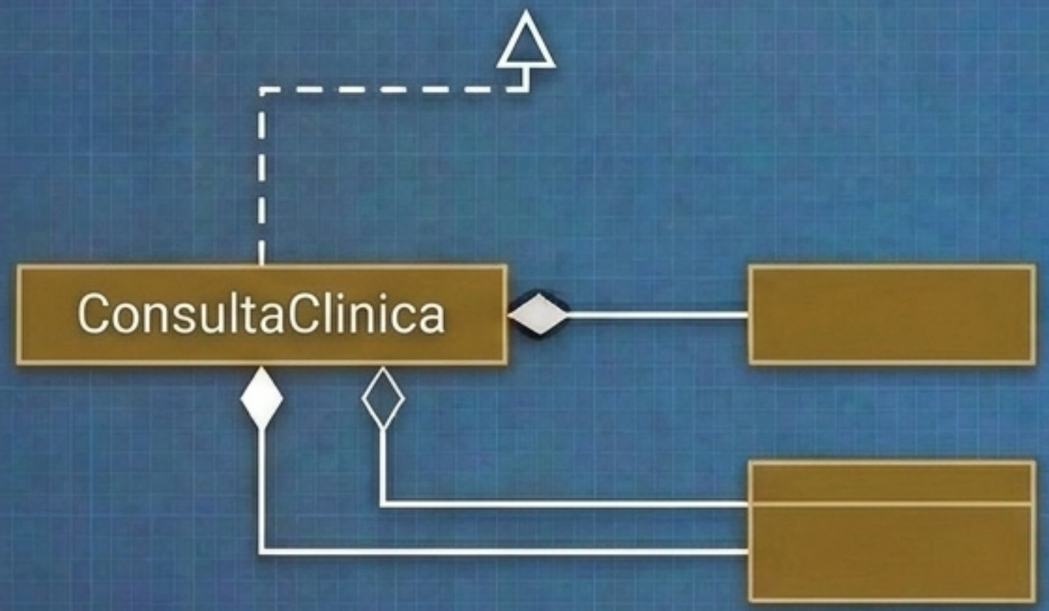
Agregação (Vida independente): O tutor e o animal existem no sistema antes e depois da consulta clínica.

Contexto: Entender o ciclo de vida dos objetos previne vazamentos de memória e inconsistências em bancos de dados.

Gemini Notebook

O Centro Nervoso do Sistema: ConsultaClinica

Design



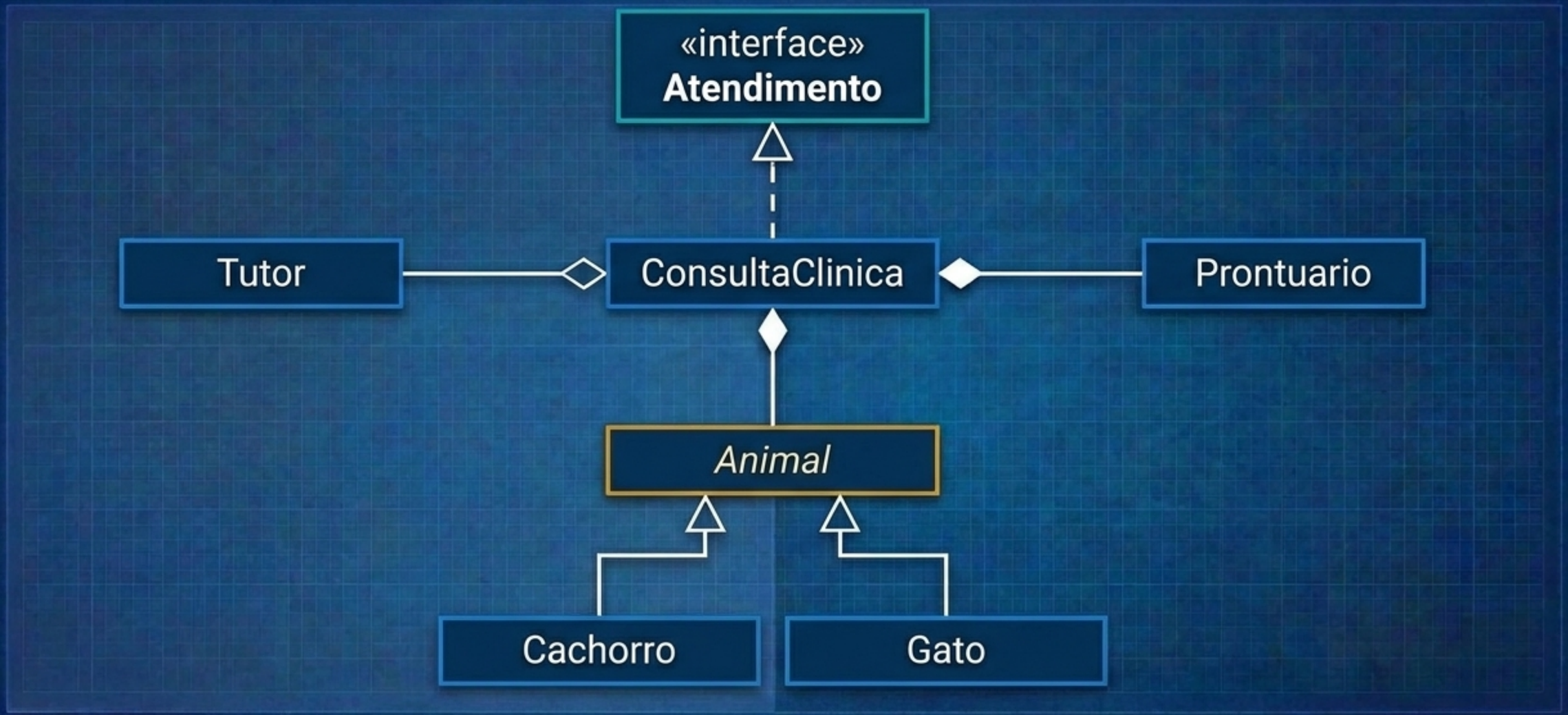
Implementação

```java
public class ConsultaClinica implements Atendimento {
    private Tutor tutor;        // Agregação
    private Animal paciente;    // Agregação
    private Prontuario pront;   // Composição

    @Override
    public double calcularValor() {
        /* lógica financeira */
        return 0.0;
    }
}
```

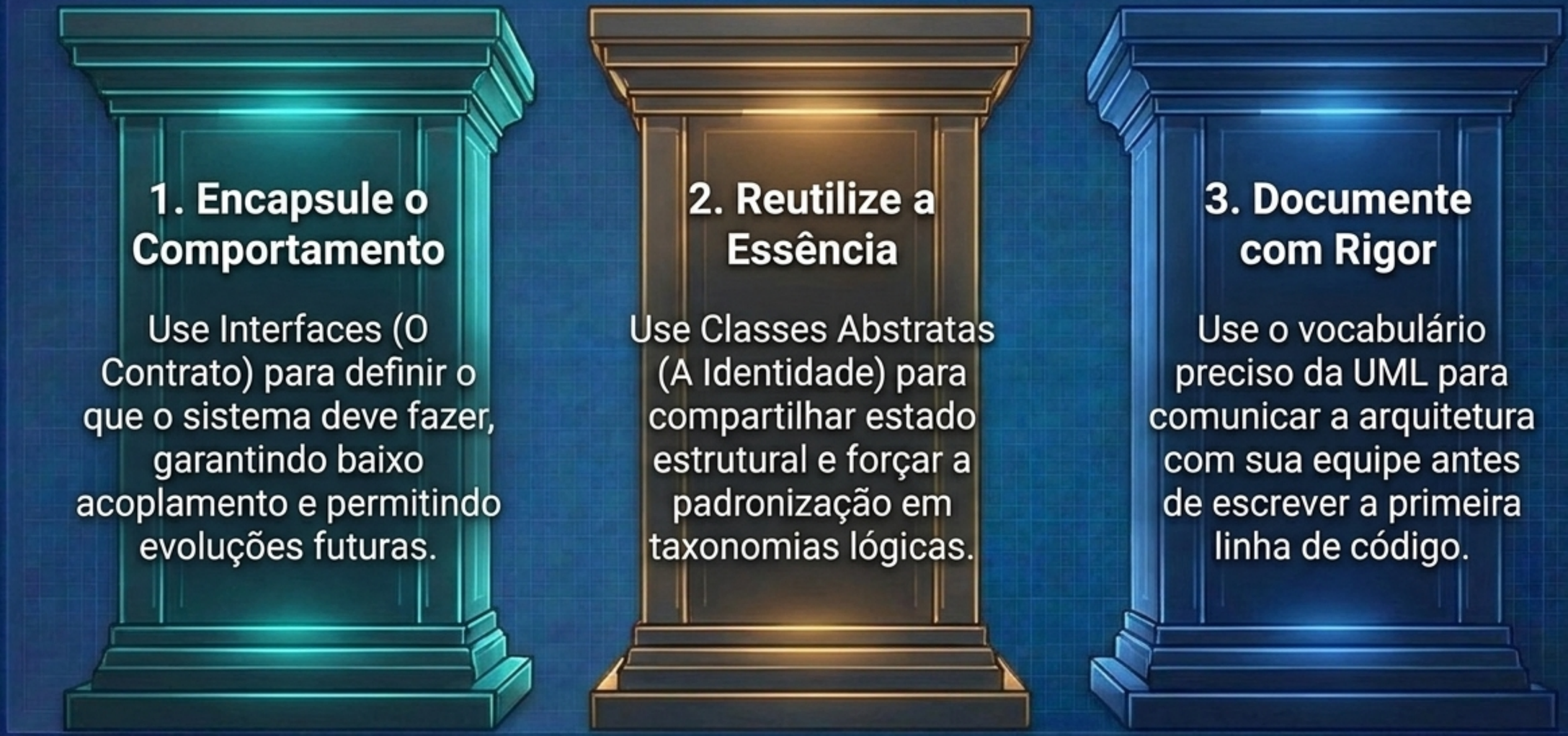
Gemini Notebook

A Visão Arquitetônica Completa



Gemini Notebook

Dominando a Arquitetura Orientada a Objetos



1. Encapsule o Comportamento

Use Interfaces (O Contrato) para definir o que o sistema deve fazer, garantindo baixo acoplamento e permitindo evoluções futuras.

2. Reutilize a Essência

Use Classes Abstratas (A Identidade) para compartilhar estado estrutural e forçar a padronização em taxonomias lógicas.

3. Documente com Rigor

Use o vocabulário preciso da UML para comunicar a arquitetura com sua equipe antes de escrever a primeira linha de código.

Gemini Notebook